

NLP with Transformers

Unit 4 Comprehensive Notes

Pages 15–84 | Chapters 1 (cont.) · 2 · 3

Study Notes

Contents

1	The Hugging Face Ecosystem (Ch. 1, pp. 15–20)	3
1.1	The Hugging Face Hub	3
1.2	Hugging Face Tokenizers	5
1.3	Hugging Face Datasets	5
1.4	Hugging Face Accelerate	6
1.5	Main Challenges with Transformers	6
2	Text Classification (Ch. 2, pp. 21–55)	6
2.1	Typical Pipeline Overview	7
2.2	The Dataset	7
2.2.1	Loading from the Hub	7
2.2.2	Loading Data Not on the Hub	8
2.2.3	From Datasets to DataFrames	9
2.2.4	Exploratory Data Analysis	9
2.3	From Text to Tokens	10
2.3.1	Character Tokenisation	10
2.3.2	Word Tokenisation	11
2.3.3	Subword Tokenisation	11
2.3.4	Tokenising the Whole Dataset	12
2.4	Training a Text Classifier	13
2.4.1	Approach 1 — Transformers as Feature Extractors	14
2.4.2	Approach 2 — Fine-Tuning DistilBERT	17
2.4.3	TensorFlow / Keras Alternative	19
2.4.4	Error Analysis	19
2.4.5	Saving and Sharing the Model	20
2.5	Chapter 2 Summary	21
3	Transformer Anatomy (Ch. 3, pp. 57–84)	21
3.1	The Transformer Architecture	21
3.1.1	Taxonomy of Transformer Models	22
3.2	The Encoder	23
3.3	Self-Attention	23
3.3.1	Scaled Dot-Product Attention	24
3.3.2	Multi-Head Attention	25

3.4	The Feed-Forward Layer	27
3.5	Layer Normalisation and Skip Connections	27
3.6	Positional Embeddings	28
3.7	Full Transformer Encoder	29
3.8	Adding a Classification Head	30
3.9	The Decoder	31
3.10	Interoperability Between Frameworks	31
3.11	The Transformer Family Tree	33
3.11.1	Encoder-Only Branch	33
3.11.2	Decoder-Only Branch	34
3.11.3	Encoder-Decoder Branch	34
3.12	Chapter 3 Summary — Building Blocks	35
4	Quick-Reference Cheat Sheet	35
4.1	Important Hyperparameters for DistilBERT / BERT	35
4.2	Fine-Tuning Checklist	36
4.3	Key Equations	36

1 The Hugging Face Ecosystem (Ch. 1, pp. 15–20)

Big picture. Training a model is only one piece of an NLP project. Efficiently processing data, sharing results, and ensuring reproducibility are equally important. The Hugging Face ecosystem addresses all of these needs through a family of tightly integrated libraries and the Hub.

The Hugging Face ecosystem has two main parts:

1. **A family of libraries** — providing code for tokenisation, training, data loading, and distributed training.
2. **The Hub** — providing pretrained model weights, datasets, evaluation metric scripts, and interactive inference widgets.

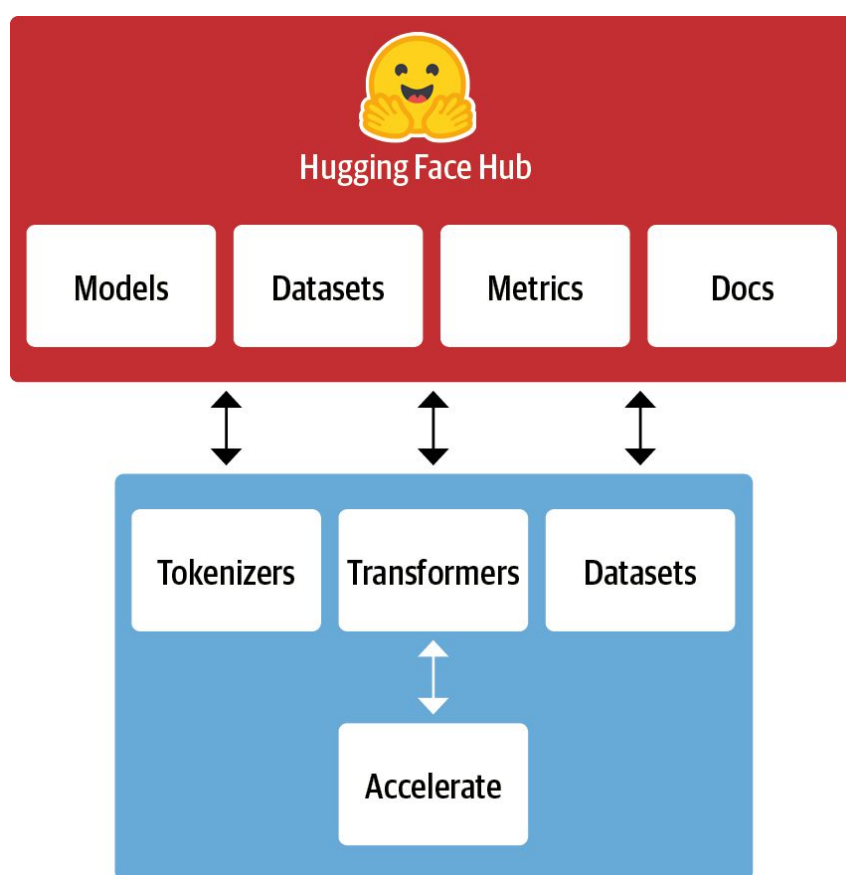


Figure 1: Overview of the Hugging Face ecosystem: the Hub (top, red) hosts models, datasets, metrics, and docs; the libraries (blue) include Tokenizers, Transformers, Datasets, and Accelerate.

1.1 The Hugging Face Hub

Transfer learning is a key driver of transformer success — it allows pretrained models to be reused for new tasks, dramatically reducing the labelled data and compute required.

- The Hub hosts **over 20,000 freely available models** (at time of writing; the number has grown substantially since).

- Filters for *task*, *framework*, and *dataset* make it easy to find promising candidates.
- Loading a model in code is a single line:

```
1 from transformers import pipeline
2 classifier = pipeline("text-classification",
3                       model="distilbert-base-uncased")
```

Listing 1: Loading a model from the Hub

- The Hub also hosts **datasets** and **metric scripts** so that published results can be reproduced.
- **Model cards** and **dataset cards** document contents and intended use, helping practitioners make informed decisions.
- Every model on the Hub can be tested interactively through **task-specific inference widgets** — no code required.

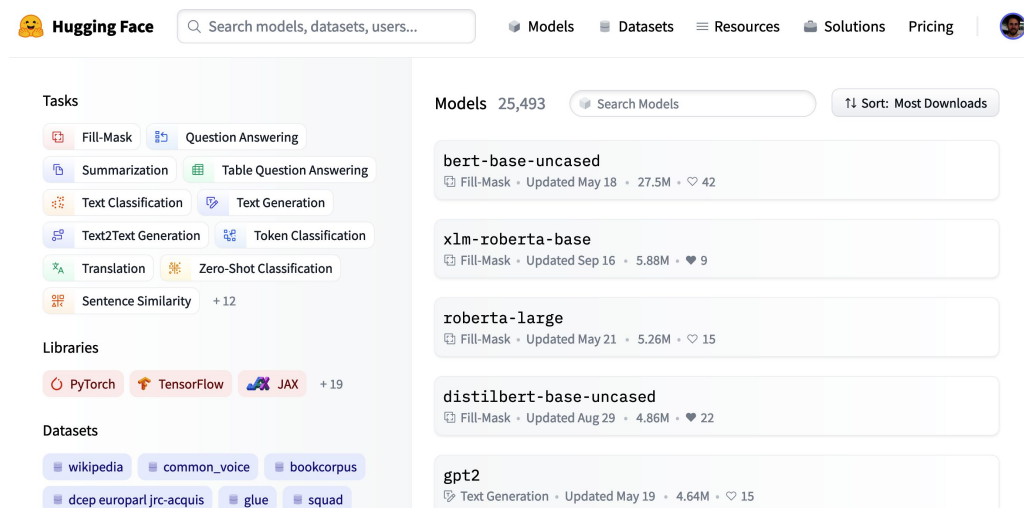


Figure 2: The Models page of the Hugging Face Hub, with task/framework/dataset filters on the left and a ranked model list on the right.

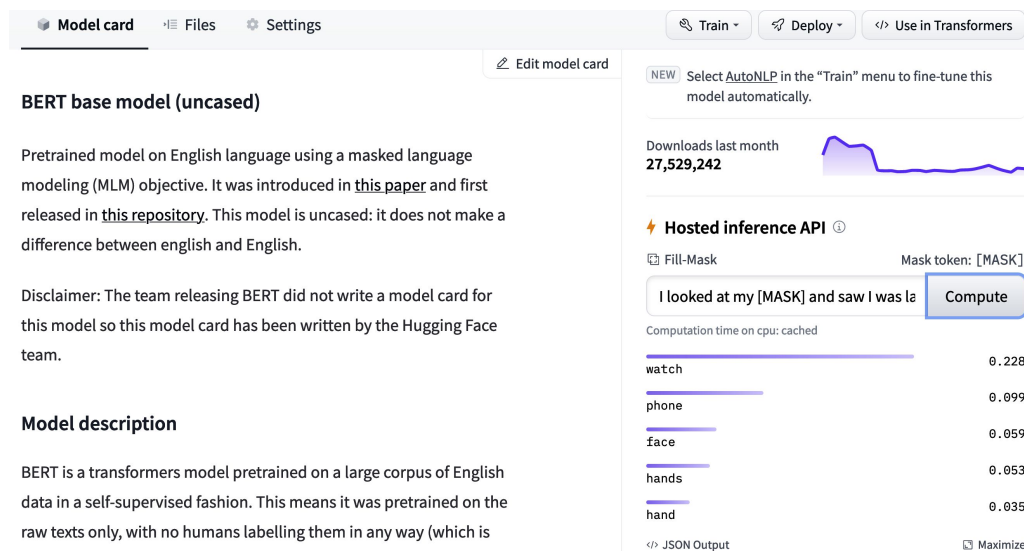


Figure 3: An example model card (BERT base uncased) showing the inference widget on the right, which lets you test fill-mask predictions directly in the browser.

Tip. PyTorch and TensorFlow each maintain their own model hubs. If a model or dataset is not on the Hugging Face Hub, it is worth checking those resources too.

1.2 Hugging Face Tokenizers

Every NLP pipeline includes a **tokenisation** step that splits raw text into smaller units called **tokens** (words, sub-words, or individual characters). Transformer models then consume *numerical representations* of these tokens.

Key features of the Tokenizers library:

- Implements many tokenisation strategies (WordPiece, BPE, Unigram, ...).
- Extremely fast thanks to its **Rust backend**.
- Handles all pre- and post-processing: input normalisation, special-token insertion, output formatting.
- Tokenisers are loaded in exactly the same way as pretrained model weights:

```
1 from transformers import AutoTokenizer
2 tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
```

Listing 2: Loading a pretrained tokenizer

1.3 Hugging Face Datasets

- Provides a **standard interface** for thousands of datasets available on the Hub.
- **Smart caching** avoids repeating preprocessing between runs.

- Avoids RAM limitations via **memory mapping** — file contents are stored in virtual memory; multiple processes can read/write efficiently.
- Interoperable with **Pandas** and **NumPy**.
- Ships **metric scripts** for many standard NLP benchmarks, making results reproducible and trustworthy.

1.4 Hugging Face Accelerate

Accelerate adds a **layer of abstraction** over normal PyTorch training loops, handling all the infrastructure-specific logic (single GPU, multi-GPU, TPU, mixed-precision, ...) automatically. The same training script runs on a laptop and on a multi-node cluster with minimal changes.

1.5 Main Challenges with Transformers

Challenge	Description
Language	NLP research is dominated by English. Pretrained models for rare or low-resource languages are scarce. (<i>Addressed in Ch. 4 with multilingual transformers.</i>)
Data availability	Transfer learning reduces labelled data requirements, but models still need more labelled examples than a human does for the same task. (<i>Ch. 9</i>)
Long documents	Self-attention scales quadratically with sequence length, making very long documents expensive. (<i>Ch. 11</i>)
Opacity	It is hard to explain <i>why</i> a transformer made a particular prediction — a problem in high-stakes deployments. (<i>Ch. 2 & 4</i>)
Bias	Models pretrained on internet text absorb the biases present in that data (racial, gender, and other biases). (<i>Ch. 10</i>)

2 Text Classification (Ch. 2, pp. 21–55)

Chapter goal. Build an emotion classifier for English tweets using a variant of BERT called **DistilBERT**. Two complementary approaches are explored: *feature extraction* (frozen backbone) and *fine-tuning* (end-to-end training).

Text classification is one of the most common NLP tasks. Applications:

- Tagging customer feedback into product categories.
- Routing support tickets by language or topic.
- Spam filtering.

- **Sentiment analysis** — identifying the polarity (positive / negative) of a text.



Figure 4: Example tweet about a Tesla car roof — a company might use sentiment analysis on such posts to gauge customer opinion.

The chapter uses the **CARER emotion dataset**: six basic emotions (*anger*, *disgust*, *fear*, *joy*, *sadness*, *surprise*) extracted from English tweets.

2.1 Typical Pipeline Overview

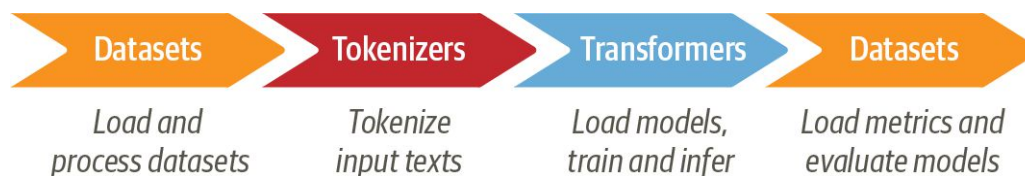


Figure 5: A typical pipeline for training transformer models: Datasets → Tokenizers → Transformers (load, train, infer) → Datasets (metrics, evaluate).

The pipeline uses three core Hugging Face libraries in sequence:

1. **Datasets** — load and process the dataset.
2. **Tokenizers** — tokenise input texts.
3. **Transformers** — load pretrained DistilBERT, train, and run inference.

The chapter uses **DistilBERT**: achieves $\approx 97\%$ of BERT’s performance while being 40% smaller and 60% faster. A **checkpoint** is the set of weights loaded into a given transformer architecture.

2.2 The Dataset

2.2.1 Loading from the Hub

```

1 from datasets import load_dataset
2 emotions = load_dataset("emotion")
3 emotions
4 # DatasetDict({
5 #   train:      Dataset({features: ['text', 'label'],
6 #     num_rows: 16000})
7 #   validation: Dataset({features: ['text', 'label'],
8 #     num_rows: 2000})
9 #   test:       Dataset({features: ['text', 'label'],
10 #     num_rows: 2000})
11 # })

```

Listing 3: Loading the emotion dataset

A `DatasetDict` behaves like a Python dict; each split is a `Dataset` object that behaves like a Python list/array:

```

1 train_ds = emotions["train"]
2 len(train_ds)           # 16000
3 train_ds[0]             # {'label': 0, 'text': 'i didnt feel
4                          #   humiliated'}
5 train_ds.column_names   # ['text', 'label']
6 print(train_ds.features)
7 # {'text': Value(dtype='string'), 'label': ClassLabel(
8 #   num_classes=6,
9 #   names=['sadness', 'joy', 'love', 'anger', 'fear', 'surprise'])}

```

Listing 4: Accessing dataset rows

Under the hood. Datasets is backed by **Apache Arrow** — a typed columnar format that is more memory-efficient than native Python. This enables slicing, batched access, and interoperability with Pandas and NumPy without loading everything into RAM.

2.2.2 Loading Data Not on the Hub

Format	Script	Example
CSV	csv	<code>load_dataset("csv", data_files="my_file.csv")</code>
Text	text	<code>load_dataset("text", data_files="my_file.txt")</code>
JSON	json	<code>load_dataset("json", data_files="my_file.jsonl")</code>

```

1 emotions_local = load_dataset("csv", data_files="train.txt",
2                               sep=";", names=["text", "label"])
3 # Or directly from a URL:
4 dataset_url = "https://www.dropbox.com/.../train.txt?dl=1"
5 emotions_remote = load_dataset("csv", data_files=dataset_url,
6                                sep=";", names=["text", "label"])

```

Listing 5: Loading a CSV/text file with custom delimiter

2.2.3 From Datasets to DataFrames

```

1 import pandas as pd
2 emotions.set_format(type="pandas")
3 df = emotions["train"][:]
4 df.head()
5 # Converts integer labels to names:
6 def label_int2str(row):
7     return emotions["train"].features["label"].int2str(row)
8 df["label_name"] = df["label"].apply(label_int2str)

```

Listing 6: Converting to Pandas for visualisation

`set_format()` changes the *output format* only; the underlying Arrow table is unchanged. Call `reset_format()` when done with Pandas operations.

2.2.4 Exploratory Data Analysis

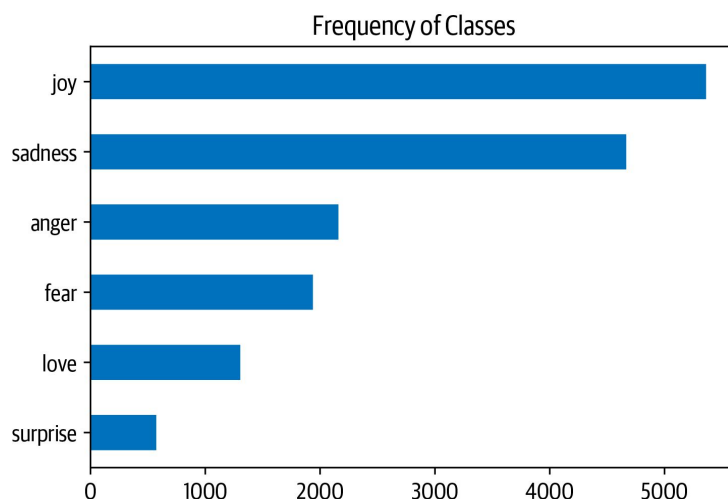


Figure 6: Frequency of emotion classes in the training set. The dataset is heavily imbalanced: *joy* and *sadness* are 5–10× more common than *love* and *surprise*.

Class distribution. Strategies for handling imbalanced datasets:

1. Randomly **oversample** the minority class.
2. Randomly **undersample** the majority class.
3. **Gather more labelled data** from under-represented classes.

Important. Apply sampling *after* creating train/test splits to prevent data leakage.

Tweet length analysis

```

1 df["Words Per Tweet"] = df["text"].str.split().apply(len)
2 df.boxplot("Words Per Tweet", by="label_name",
3           grid=False, showfliers=False, color="black")

```

Listing 7: Computing words-per-tweet distribution

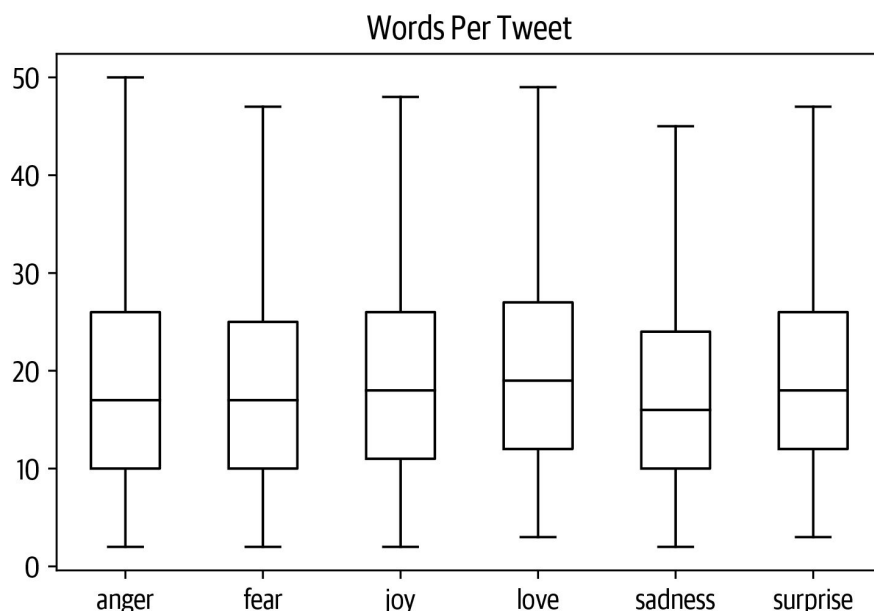


Figure 7: Box plot of words per tweet broken down by emotion class. Most tweets contain ≈ 15 words — well within DistilBERT’s 512-token context window, so truncation will not be a problem.

DistilBERT context limit. The **maximum context size** is 512 tokens. Texts longer than this must be *truncated*, which can degrade performance if the truncated content contains crucial information.

2.3 From Text to Tokens

Transformer models cannot accept raw strings. Text must be **tokenised** (split into atomic units) and **numericalized** (mapped to integers), then typically encoded as numerical vectors.

2.3.1 Character Tokenisation

Split text into individual characters; map each to a unique integer.

```

1 text = "Tokenizing text is a core task of NLP."
2 tokenized_text = list(text)
3 # Build vocabulary
4 token2idx = {ch: idx for idx, ch in
5             enumerate(sorted(set(tokenized_text)))}
6 # Numericalize
7 input_ids = [token2idx[t] for t in tokenized_text]

```

Listing 8: Character-level tokenisation in Python

One-hot encoding. Each token ID is then mapped to a sparse binary vector of length $|\mathcal{V}|$ (vocabulary size) with a single “hot” entry:

```
1 import torch
2 import torch.nn.functional as F
3 input_ids = torch.tensor(input_ids)
4 one_hot_encodings = F.one_hot(input_ids, num_classes=len(
    token2idx))
5 # Shape: [n_tokens, vocab_size] e.g. torch.Size([38, 20])
```

Listing 9: One-hot encoding with PyTorch

Always set `num_classes` in `F.one_hot()`; without it the vectors may be shorter than the vocabulary length. TensorFlow equivalent: `tf.one_hot(depth=num_classes)`.

Limitations of character tokenisation:

- Any linguistic structure (words, morphemes) must be *learned from scratch* — expensive in compute, memory, and data.
- Rarely used in modern NLP pipelines.

2.3.2 Word Tokenisation

Split on whitespace; map each word to an integer.

```
1 tokenized_text = text.split()
2 # ['Tokenizing', 'text', 'is', 'a', 'core', 'task', 'of', 'NLP', '.']
```

Listing 10: Whitespace tokenisation

Problems:

- Punctuation is attached to words (NLP. $\neq$ NLP).
- Declinations, conjugations, misspellings explode the vocabulary (potentially millions of entries).
- A weight matrix mapping 1M words to a 1,000-dim space has 10^9 weights — comparable to the full GPT-2 model.
- Rare words are replaced by [UNK], losing potentially important information.

2.3.3 Subword Tokenisation

The best of both worlds: splits *rare* words into smaller units while keeping *frequent* words intact. The splitting rules are *learned* from the pretraining corpus using statistical algorithms.

WordPiece (used by BERT and DistilBERT)

```

1 from transformers import AutoTokenizer
2 model_ckpt = "distilbert-base-uncased"
3 tokenizer = AutoTokenizer.from_pretrained(model_ckpt)
4
5 text = "Tokenizing text is a core task of NLP."
6 encoded_text = tokenizer(text)
7 # {'input_ids': [101, 19204, 6026, ..., 102],
8 #   'attention_mask': [1, 1, ..., 1]}
9
10 tokens = tokenizer.convert_ids_to_tokens(encoded_text.input_ids
    )
11 # ['[CLS]', 'token', '##izing', 'text', 'is', 'a', 'core',
12 #   'task', 'of', 'nl', '##p', '.', '[SEP]']

```

Listing 11: Loading the DistilBERT tokeniser and tokenising a sentence

Key observations:

1. **Special tokens** [CLS] and [SEP] are added at the start and end.
2. Tokens are **lowercased** by the base-uncased checkpoint.
3. The **##** prefix indicates a **continuation token** (no preceding whitespace); strip this prefix when converting tokens back to strings.
4. “Tokenizing” → token + ##izing; “NLP” → nl + ##p.

```

1 tokenizer.vocab_size           # 30522
2 tokenizer.model_max_length    # 512
3 tokenizer.model_input_names   # ['input_ids', 'attention_mask']

```

Listing 12: Tokeniser metadata

Token	[PAD]	[UNK]	[CLS]	[SEP]	[MASK]
ID	0	100	101	102	103

The **AutoTokenizer** class belongs to a family of *auto* classes that automatically retrieve model configuration, weights, or vocabulary from a Hub checkpoint name — making it trivial to switch between models.

2.3.4 Tokenising the Whole Dataset

```

1 def tokenize(batch):
2     return tokenizer(batch["text"], padding=True, truncation=
        True)
3
4 # batched=True encodes in batches; batch_size=None = full
   dataset in one pass
5 emotions_encoded = emotions.map(tokenize, batched=True,
    batch_size=None)

```

```

6 print(emotions_encoded["train"].column_names)
7 # ['attention_mask', 'input_ids', 'label', 'text']

```

Listing 13: Tokenising the entire corpus with map()

Padding and masking. padding=True pads shorter sequences with zeros ([PAD] tokens). The **attention mask** (a binary array of 1s and 0s) tells the model to *ignore* the padded positions so that they do not corrupt the attention computations.

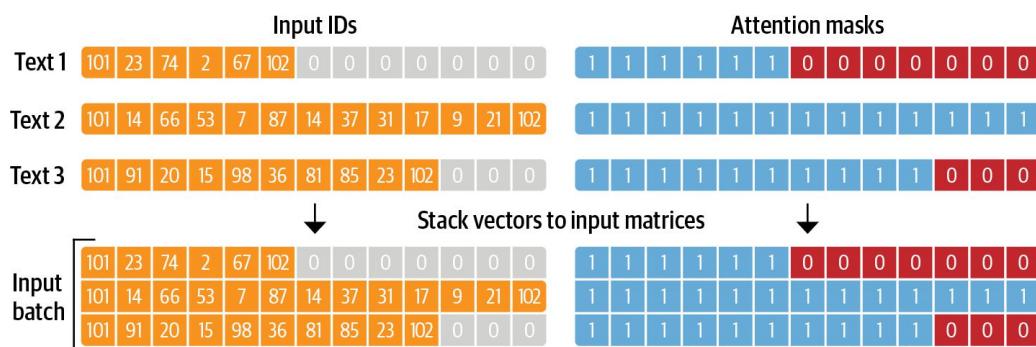


Figure 8: Padding and attention masking. Each sequence is padded to the maximum length in the batch; padded positions receive mask value 0 so the model ignores them.

2.4 Training a Text Classifier

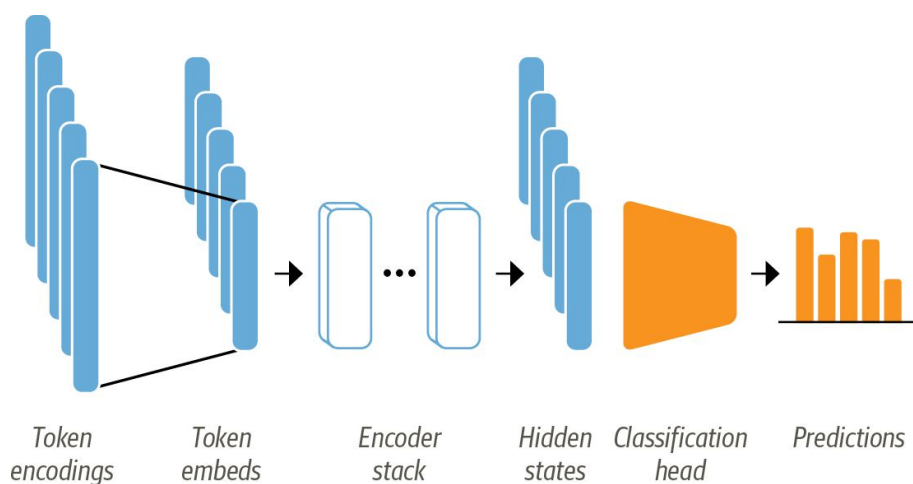


Figure 9: Architecture for sequence classification with an encoder-based transformer: token encodings → embeddings → encoder stack → hidden states → classification head → predictions.

For text classification the language-modelling head of a pretrained transformer is **replaced by a classification head**. Two training strategies are available:

Approach	What is trained?	When to use?
Feature extraction	Only the classification head; the backbone is frozen.	Limited compute, no GPU, or a quick baseline.
Fine-tuning	The entire model end-to-end.	Best accuracy; requires GPU and more time.

2.4.1 Approach 1 — Transformers as Feature Extractors

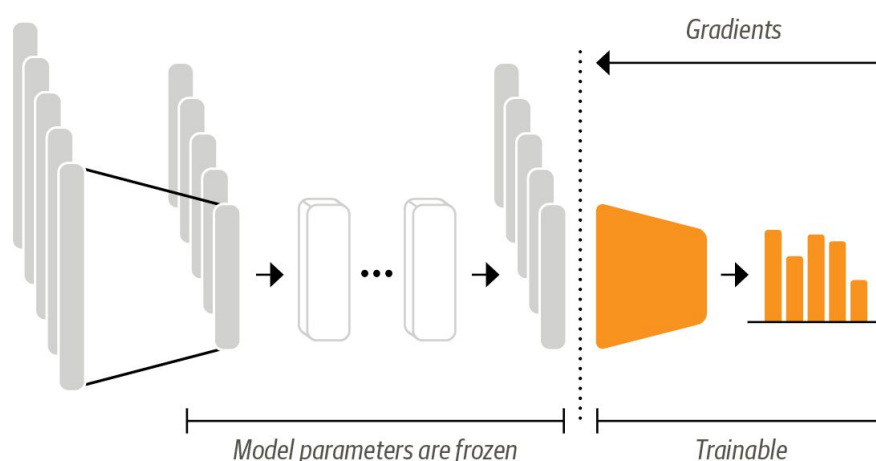


Figure 10: Feature-based approach: the DistilBERT model is frozen; only the classification head (right, orange) receives gradient updates.

Step 1: Load the pretrained body.

```

1 from transformers import AutoModel
2 import torch
3 model_ckpt = "distilbert-base-uncased"
4 device = torch.device("cuda" if torch.cuda.is_available() else
5                       "cpu")
6 model = AutoModel.from_pretrained(model_ckpt).to(device)

```

Listing 14: Loading DistilBERT for feature extraction

Step 2: Extract hidden states.

The output of `AutoModel` has shape $[\text{batch_size}, n_{\text{tokens}}, d_{\text{hidden}}]$ (e.g., $[1, 6, 768]$ for a 6-token sequence with DistilBERT).

For classification, use the hidden state of the [CLS] token (index 0):

```

1 with torch.no_grad():
2     outputs = model(**inputs)
3     # outputs.last_hidden_state shape: [batch, n_tokens, 768]
4     cls_vector = outputs.last_hidden_state[:, 0] # shape: [batch,
5                                                  768]

```

Listing 15: Extracting the [CLS] hidden state

```

1 def extract_hidden_states(batch):
2     inputs = {k: v.to(device) for k, v in batch.items()
3               if k in tokenizer.model_input_names}
4     with torch.no_grad():
5         last_hidden_state = model(**inputs).last_hidden_state
6     return {"hidden_state": last_hidden_state[:, 0].cpu().numpy
7           ()}
8
9 emotions_encoded.set_format("torch",
10                             columns=["input_ids", "attention_mask", "label"])
11 emotions_hidden = emotions_encoded.map(extract_hidden_states,
12                                       batched=True)

```

Listing 16: Extracting hidden states for the entire dataset

Step 3: Visualise with UMAP.

Project the 768-dimensional vectors to 2D using UMAP + MinMaxScaler to inspect class separability before training.

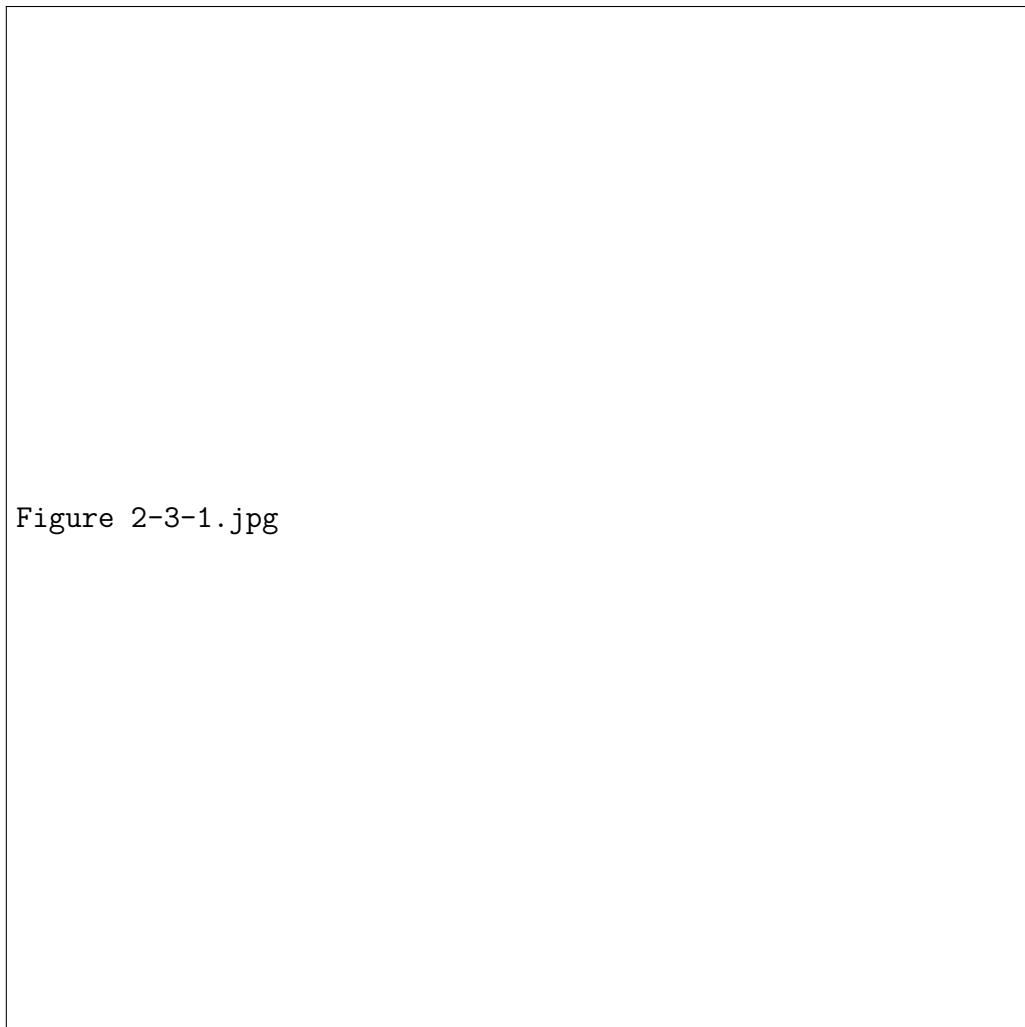


Figure 11: UMAP 2-D projection of DistilBERT hidden states coloured by emotion class. *Sadness*, *anger*, and *fear* cluster together; *joy* and *love* share a separate region; *surprise* is diffuse.

Step 4: Train a logistic regression classifier.

```

1 import numpy as np
2 from sklearn.linear_model import LogisticRegression
3 X_train = np.array(emotions_hidden["train"]["hidden_state"])
4 y_train = np.array(emotions_hidden["train"]["label"])
5 X_valid = np.array(emotions_hidden["validation"]["hidden_state"]
6                    ])
7 y_valid = np.array(emotions_hidden["validation"]["label"])
8 # X_train.shape => (16000, 768)
9
10 lr_clf = LogisticRegression(max_iter=3000)
11 lr_clf.fit(X_train, y_train)
12 lr_clf.score(X_valid, y_valid)    # ~0.633
13 # Dummy (always-predict-majority) baseline: ~0.352

```

Listing 17: Training a Scikit-learn logistic regression on frozen embeddings

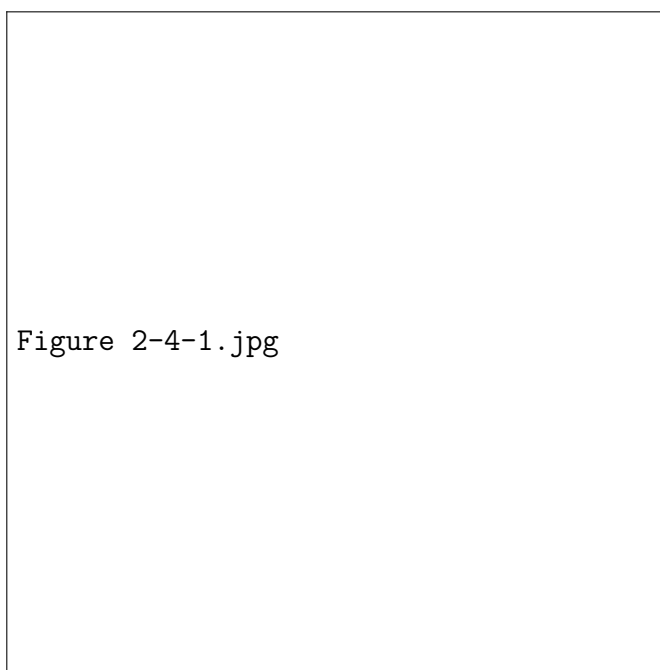
Confusion matrix (feature extraction):

Figure 12: Normalised confusion matrix for the feature-extraction approach. Accuracy $\approx 63\%$. *Anger* and *fear* are most often confused with *sadness*; *love* and *surprise* are frequently mistaken for *joy*.

2.4.2 Approach 2 — Fine-Tuning DistilBERT

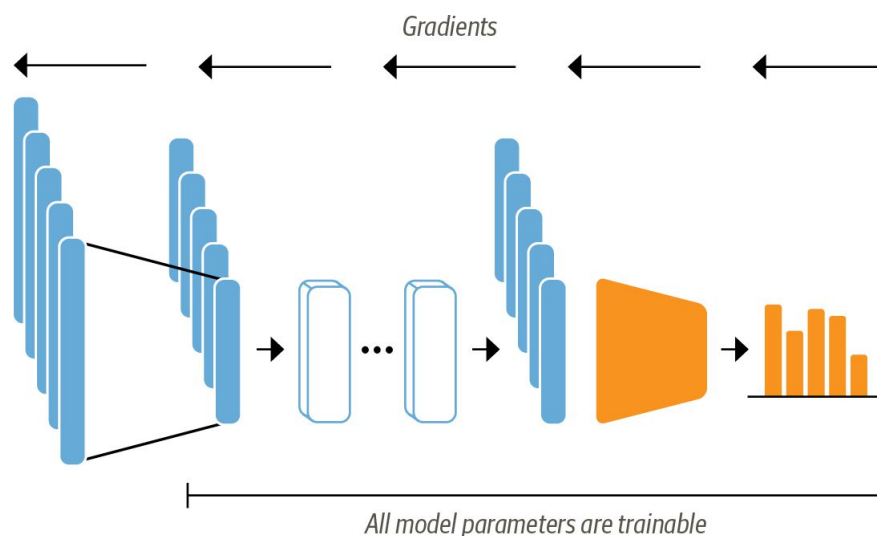


Figure 13: Fine-tuning approach: the entire DistilBERT model plus the classification head are trained end-to-end. Gradients flow back through all layers.

Step 1: Load the model with a classification head.

```
1 from transformers import AutoModelForSequenceClassification
2 num_labels = 6
3 model = (AutoModelForSequenceClassification
4         .from_pretrained(model_ckpt, num_labels=num_labels)
5         .to(device))
```

Listing 18: Loading AutoModelForSequenceClassification

Step 2: Define metrics.

```
1 from sklearn.metrics import accuracy_score, f1_score
2
3 def compute_metrics(pred):
4     labels = pred.label_ids
5     preds = pred.predictions.argmax(-1)
6     f1 = f1_score(labels, preds, average="weighted")
7     acc = accuracy_score(labels, preds)
8     return {"accuracy": acc, "f1": f1}
```

Listing 19: Defining compute_metrics for the Trainer

Step 3: Configure and run the Trainer.

```
1 from transformers import Trainer, TrainingArguments
2
3 batch_size = 64
4 logging_steps = len(emotions_encoded["train"]) // batch_size
5 model_name = f"{model_ckpt}-finetuned-emotion"
6
7 training_args = TrainingArguments(
8     output_dir=model_name,
```

```

9     num_train_epochs=2,
10    learning_rate=2e-5,
11    per_device_train_batch_size=batch_size,
12    per_device_eval_batch_size=batch_size,
13    weight_decay=0.01,
14    evaluation_strategy="epoch",
15    logging_steps=logging_steps,
16    push_to_hub=True,
17 )
18
19 trainer = Trainer(
20     model=model,
21     args=training_args,
22     compute_metrics=compute_metrics,
23     train_dataset=emotions_encoded["train"],
24     eval_dataset=emotions_encoded["validation"],
25     tokenizer=tokenizer,
26 )
27 trainer.train()

```

Listing 20: Training with the Hugging Face Trainer API

Epoch	Train Loss	Val Loss	Accuracy	F1
1	0.8409	0.3274	89.65%	89.23%
2	0.2550	0.2205	92.25%	92.26%

Fine-tuning achieves $\approx 92\%$ **F1**, a major improvement over the $\approx 63\%$ accuracy of the frozen feature extractor.

Confusion matrix (fine-tuning):

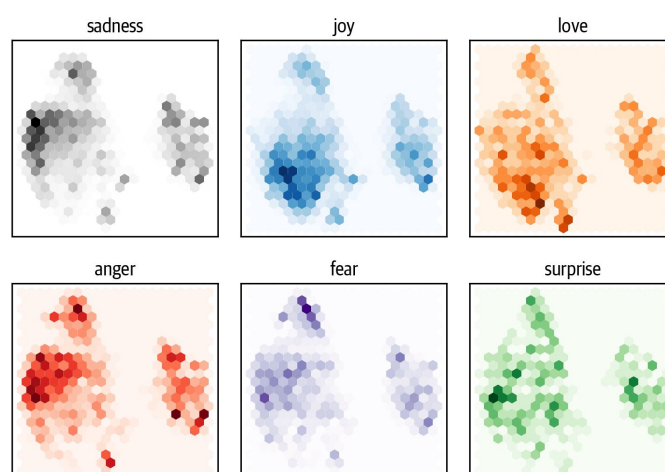


Figure 14: Normalised confusion matrix after fine-tuning. Much closer to the ideal diagonal; *love* still sometimes confused with *joy* (natural semantic overlap); *surprise* occasionally confused with *fear*.

2.4.3 TensorFlow / Keras Alternative

```

1 from transformers import TFAutoModelForSequenceClassification
2 import tensorflow as tf
3
4 tf_model = TFAutoModelForSequenceClassification.from_pretrained(
5     model_ckpt, num_labels=num_labels)
6
7 tf_train_dataset = emotions_encoded["train"].to_tf_dataset(
8     columns=tokenizer.model_input_names,
9     label_cols=["label"], shuffle=True, batch_size=batch_size)
10 tf_eval_dataset = emotions_encoded["validation"].to_tf_dataset(
11     columns=tokenizer.model_input_names,
12     label_cols=["label"], shuffle=False, batch_size=batch_size)
13
14 tf_model.compile(
15     optimizer=tf.keras.optimizers.Adam(learning_rate=5e-5),
16     loss=tf.keras.losses.SparseCategoricalCrossentropy(
17         from_logits=True),
18     metrics=tf.metrics.SparseCategoricalAccuracy())
19 tf_model.fit(tf_train_dataset, validation_data=tf_eval_dataset,
20             epochs=2)

```

Listing 21: Fine-tuning with the Keras fit() API

2.4.4 Error Analysis

Sort validation samples by loss to identify:

1. **Wrong labels** — annotation errors that can be corrected.
2. **Dataset quirks** — unusual characters or formatting that confuse the model.

```

1 from torch.nn.functional import cross_entropy
2
3 def forward_pass_with_label(batch):
4     inputs = {k: v.to(device) for k, v in batch.items()
5               if k in tokenizer.model_input_names}
6     with torch.no_grad():
7         output = model(**inputs)
8     pred_label = torch.argmax(output.logits, axis=-1)
9     loss = cross_entropy(output.logits,
10                          batch["label"].to(device), reduction="none")
11     return {"loss": pred_label.cpu().numpy(),
12            "predicted_label": pred_label.cpu().numpy()}

```

Listing 22: Computing per-sample loss for error analysis

Key findings:

- *Joy* is sometimes mislabelled in the dataset.
- The model is most confident (lowest loss) on *sadness* samples.
- Inspecting high-loss examples guides targeted dataset improvements — often more impactful than collecting more data.

2.4.5 Saving and Sharing the Model

```

1 trainer.push_to_hub(commit_message="Training completed!")
2
3 # Loading and using the published model with pipeline:
4 from transformers import pipeline
5 model_id = "transformersbook/distilbert-base-uncased-finetuned-
   emotion"
6 classifier = pipeline("text-classification", model=model_id)
7 preds = classifier("I saw a movie today and it was really good.
   ",
8                     return_all_scores=True)

```

Listing 23: Pushing the fine-tuned model to the Hub

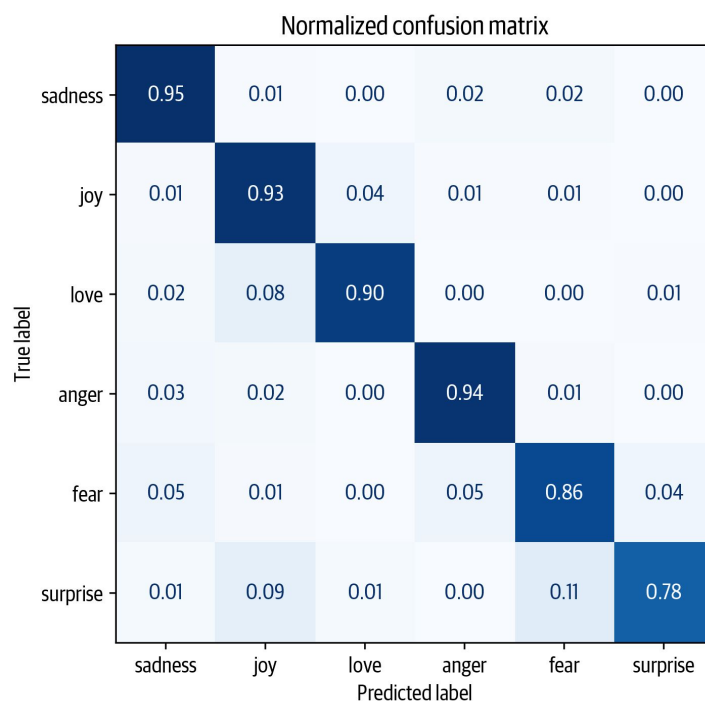


Figure 15: Class probability bar chart for the sample tweet “I saw a movie today and it was really good.” The fine-tuned model correctly predicts *joy* as the dominant emotion ($\approx 95\%$ probability).

2.5 Chapter 2 Summary

	Feature Extraction	Fine-Tuning
What is trained	Classification head only	Entire model + head
Accuracy (validation)	$\approx 63\%$	$\approx 92\%$
GPU required	No	Yes (recommended)
Training speed	Fast	Slower
Best for	Quick baseline, limited resources	Production-quality models

3 Transformer Anatomy (Ch. 3, pp. 57–84)

Chapter goal. Understand the internal mechanics of the Transformer architecture — self-attention, multi-head attention, feed-forward layers, positional embeddings — and implement them from scratch in PyTorch.

3.1 The Transformer Architecture

The original Transformer (Vaswani et al., 2017) is an **encoder-decoder** model designed for sequence-to-sequence tasks such as machine translation.

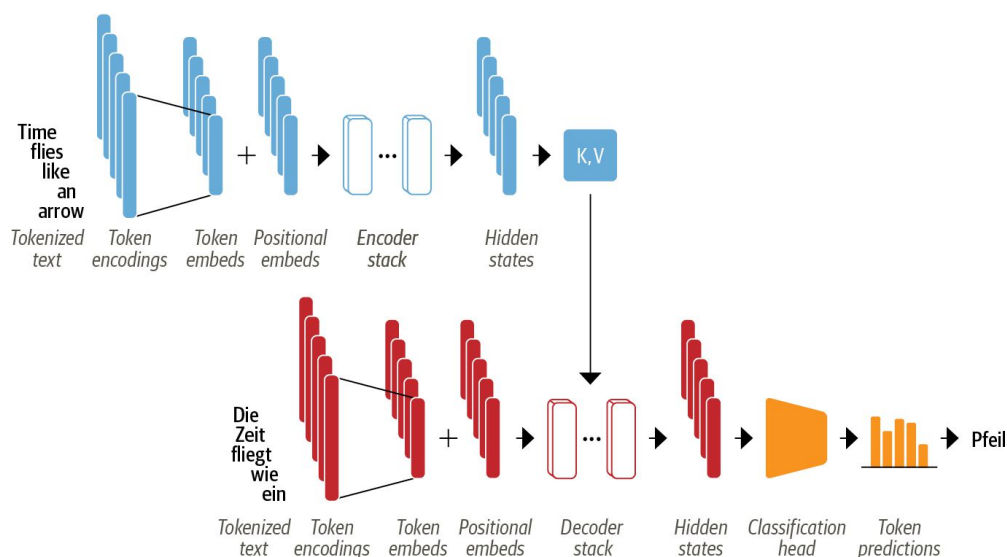


Figure 16: Encoder-decoder architecture. **Encoder** (top): tokenised input → token embeddings → positional embeddings → encoder stack → hidden states. **Decoder** (bottom): tokenised target → embeddings → decoder stack (attending to encoder outputs) → hidden states → classification head → token predictions.

Key structural observations:

- Token embeddings are combined with **positional embeddings** because the attention mechanism itself has no notion of token order.

- The encoder is a **stack of identical encoder layers**.
- The decoder is a **stack of identical decoder layers**, each of which attends to the encoder outputs.
- Decoding is **auto-regressive**: the decoder generates one token at a time, feeding each new token back as input, until [EOS] is produced or a maximum length is reached.

3.1.1 Taxonomy of Transformer Models

Type	Attention	Examples & Use cases
Encoder-only	Bidirectional (sees full context)	BERT, RoBERTa, Distil-BERT Text classification, NER, QA
Decoder-only	Causal / autoregressive (left context only)	GPT, GPT-2, GPT-3 Text generation, completion
Encoder-decoder	Bidirectional encoder + causal decoder	T5, BART, M2M-100 Translation, summarisation

Note. The boundaries are blurry. Decoder-only models (GPT family) can be prompted for translation; encoder-only models (BERT) can be applied to summarisation tasks.

3.2 The Encoder

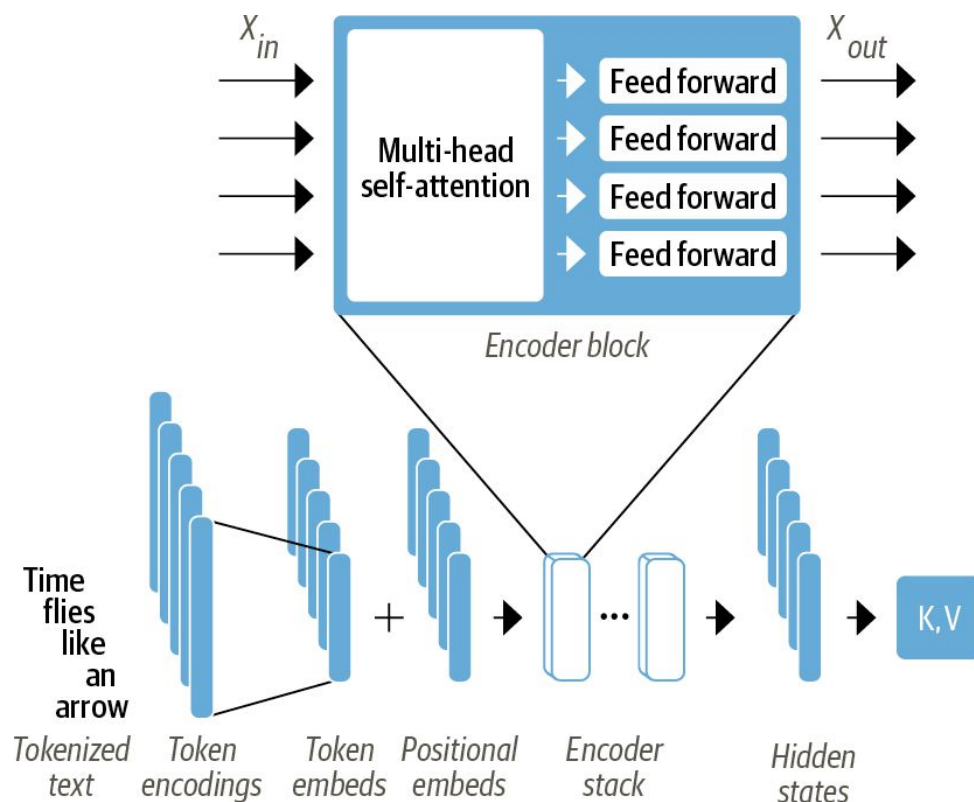


Figure 17: Zooming into a single encoder layer. Inputs pass through (1) a multi-head self-attention sublayer, then (2) a position-wise feed-forward sublayer. Each sublayer uses skip connections and layer normalisation.

Each encoder layer contains two sublayers:

1. **Multi-head self-attention layer**
2. **Position-wise feed-forward network (FFN)**

Both sublayers use **skip (residual) connections** and **layer normalisation** for stable training.

3.3 Self-Attention

Self-attention produces a new embedding $\mathbf{x}'_i$ for each token i as a *weighted average* of all token embeddings:

$$\mathbf{x}'_i = \sum_{j=1}^n w_{ji} \mathbf{x}_j, \quad \sum_j w_{ji} = 1$$

where the **attention weights** w_{ji} are learned dynamically from the content of the sequence. This allows the model to create **contextualised embeddings** — the representation of a word depends on its context.

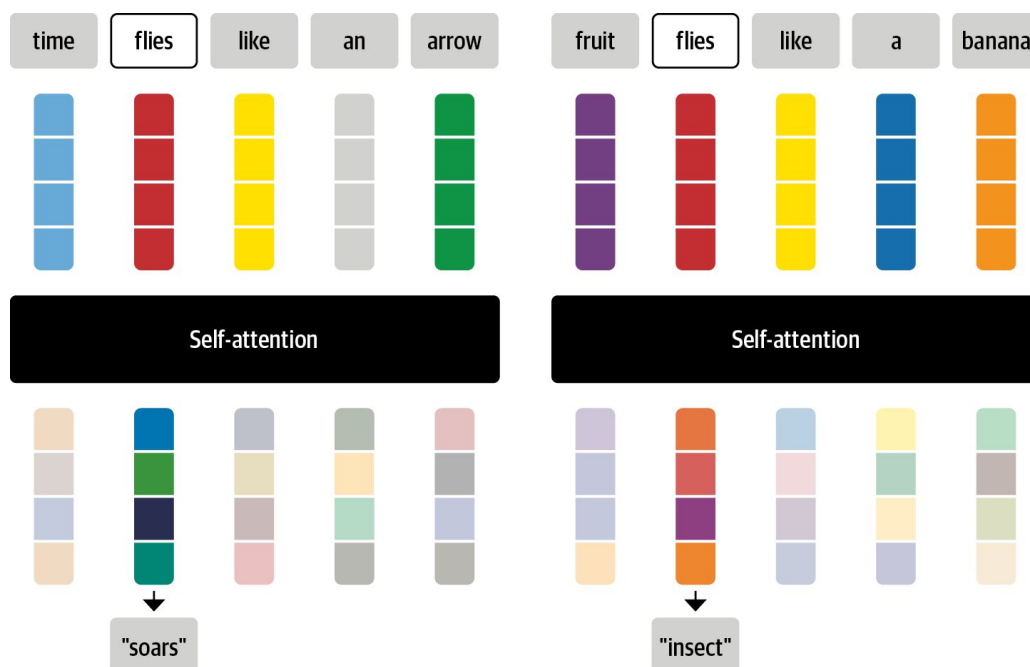


Figure 18: Self-attention in action. The same word “flies” gets two different contextualised representations depending on its sentence: *soars* (verb reading) vs. *insect* (noun reading).

3.3.1 Scaled Dot-Product Attention

The standard implementation uses four steps:

1. **Project** each token embedding into three vectors: **query** ($\mathbf{q}$), **key** ($\mathbf{k}$), **value** ($\mathbf{v}$).
2. **Compute attention scores** via the dot product:

$$\text{score}(\mathbf{q}, \mathbf{k}) = \mathbf{q}^\top \mathbf{k}$$

Result: an $n \times n$ matrix of scores.

3. **Scale and softmax:**

$$w_{ij} = \text{softmax}\left(\frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}}\right)$$

Scaling by $\sqrt{d_k}$ prevents large dot products from saturating the softmax.

4. **Weighted sum of values:**

$$\mathbf{x}'_i = \sum_j w_{ij} \mathbf{v}_j$$

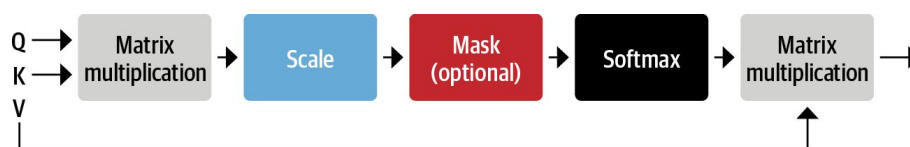


Figure 19: Operations in scaled dot-product attention: $\mathbf{Q}$ and $\mathbf{K}$ are multiplied, scaled, optionally masked, then softmaxed; the result weights the values $\mathbf{V}$.

PyTorch implementation:


```

1 import torch
2 import torch.nn.functional as F
3 from math import sqrt
4
5 def scaled_dot_product_attention(query, key, value, mask=None):
6     dim_k = query.size(-1)
7     scores = torch.bmm(query, key.transpose(1, 2)) / sqrt(dim_k)
8
9     if mask is not None:
10         scores = scores.masked_fill(mask == 0, float("-inf"))
11     weights = F.softmax(scores, dim=-1)
12     return torch.bmm(weights, value)

```

Listing 24: Scaled dot-product attention

Queries, keys, and values — an analogy. Think of a supermarket. Your recipe (the query) lists ingredients. You scan shelf labels (keys) for matches. When a label matches, you take the item (value). Self-attention is a “soft” version: every label partially matches every ingredient, so you take a weighted combination of all items.

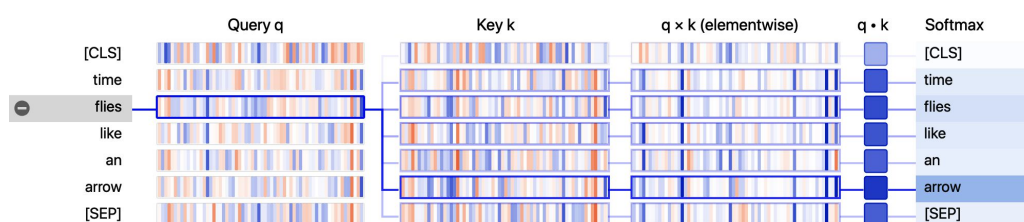


Figure 20: Attention-weight visualisation (BertViz). For each selected token (left), line intensity shows how strongly it attends to other tokens (right). “Flies” attends to “arrow” in the first sentence and to “fruit”/“banana” in the second, disambiguating its meaning.

3.3.2 Multi-Head Attention

Running *several* self-attention heads in parallel, each with its own $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$ projections, allows the model to simultaneously attend to *different aspects* of the sequence (e.g., one head for subject-verb agreement, another for nearby adjectives).

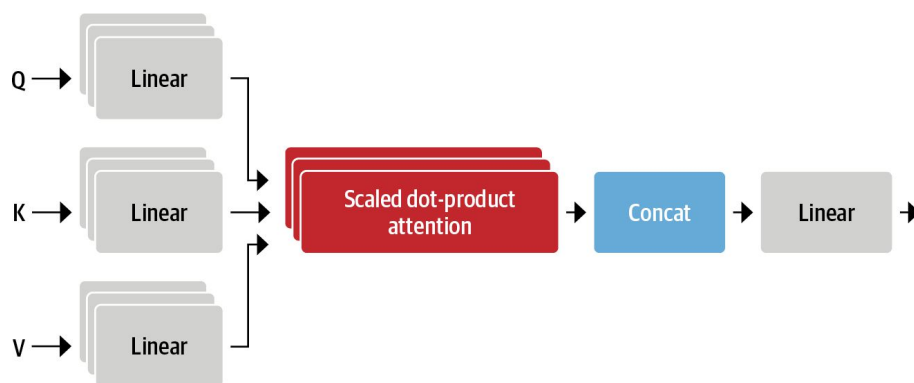


Figure 21: Multi-head attention: h parallel scaled dot-product attention operations, each with independent linear projections, whose outputs are concatenated and linearly projected to the original dimension.

PyTorch implementation:

```

1 from torch import nn
2 class AttentionHead(nn.Module):
3     def __init__(self, embed_dim, head_dim):
4         super().__init__()
5         self.q = nn.Linear(embed_dim, head_dim)
6         self.k = nn.Linear(embed_dim, head_dim)
7         self.v = nn.Linear(embed_dim, head_dim)
8
9     def forward(self, hidden_state):
10        return scaled_dot_product_attention(
11            self.q(hidden_state),
12            self.k(hidden_state),
13            self.v(hidden_state))

```

Listing 25: Single attention head

```

1 class MultiHeadAttention(nn.Module):
2     def __init__(self, config):
3         super().__init__()
4         embed_dim = config.hidden_size # 768 for BERT
5         num_heads = config.num_attention_heads # 12 for BERT
6         head_dim = embed_dim // num_heads # 64
7         self.heads = nn.ModuleList(
8             [AttentionHead(embed_dim, head_dim)
9              for _ in range(num_heads)])
10        self.output_linear = nn.Linear(embed_dim, embed_dim)
11
12    def forward(self, hidden_state):
13        x = torch.cat([h(hidden_state) for h in self.heads],
14                      dim=-1)
15        return self.output_linear(x)

```

Output shape: [batch, seq_len, embed_dim] e.g. [1, 5, 768]

Listing 26: Multi-head attention layer

3.4 The Feed-Forward Layer

A two-layer fully connected network applied *independently* to each token embedding (position-wise):

$$\text{FFN}(\mathbf{x}) = \text{GELU}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

- Hidden size of the first layer: $4 \times d_{\text{model}}$ (e.g., $4 \times 768 = 3072$ for BERT).
- Activation: **GELU** (Gaussian Error Linear Unit).
- This layer is where most **capacity and memorisation** is believed to occur; it is the part most often scaled when building larger models.

```

1 class FeedForward(nn.Module):
2     def __init__(self, config):
3         super().__init__()
4         self.linear_1 = nn.Linear(config.hidden_size,
5                                   config.intermediate_size)
6         self.linear_2 = nn.Linear(config.intermediate_size,
7                                   config.hidden_size)
8         self.gelu      = nn.GELU()
9         self.dropout   = nn.Dropout(config.hidden_dropout_prob)
10
11     def forward(self, x):
12         x = self.linear_1(x)
13         x = self.gelu(x)
14         x = self.linear_2(x)
15         return self.dropout(x)

```

Listing 27: Feed-forward sublayer

3.5 Layer Normalisation and Skip Connections

Two placements of layer normalisation are common:

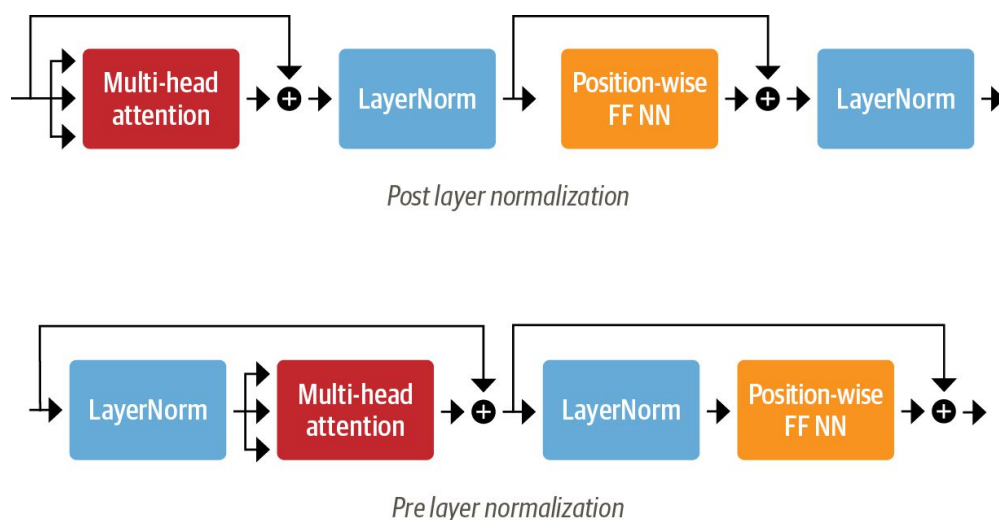


Figure 22: **Post layer normalisation** (left, original paper): LayerNorm *between* skip connections — unstable, often needs learning-rate warm-up. **Pre layer normalisation** (right, modern default): LayerNorm *inside* the skip connection span — more stable, no warm-up needed.

```

1 class TransformerEncoderLayer(nn.Module):
2     def __init__(self, config):
3         super().__init__()
4         self.layer_norm_1 = nn.LayerNorm(config.hidden_size)
5         self.layer_norm_2 = nn.LayerNorm(config.hidden_size)
6         self.attention = MultiHeadAttention(config)
7         self.feed_forward = FeedForward(config)
8
9     def forward(self, x):
10        # Self-attention + skip connection
11        hidden = self.layer_norm_1(x)
12        x = x + self.attention(hidden)
13        # FFN + skip connection
14        x = x + self.feed_forward(self.layer_norm_2(x))
15        return x

```

Listing 28: Transformer encoder layer with pre-layer normalisation

3.6 Positional Embeddings

Self-attention is **permutation equivariant**: it treats tokens as an unordered set. Positional embeddings inject order information.

Three main approaches:

Type		Description
Learnable (absolute)		An <code>nn.Embedding</code> table indexed by position — learned during pretraining. Used in BERT. Simple and effective when data is plentiful.
Sinusoidal (absolute)		Fixed modulated sine/cosine patterns. Good when data is scarce.
Relative		Encodes <i>relative</i> distances between tokens; the attention mechanism itself is modified. Used in DeBERTa. Rotary position embeddings (RoPE, used in GPT-Neo) combine absolute and relative ideas.

```

1 class Embeddings(nn.Module):
2     def __init__(self, config):
3         super().__init__()
4         self.token_embeddings = nn.Embedding(config.vocab_size,
5                                             config.hidden_size)
6         self.position_embeddings = nn.Embedding(
7             config.max_position_embeddings, config.hidden_size)
8         self.layer_norm = nn.LayerNorm(config.hidden_size, eps=1e-12)
9         self.dropout = nn.Dropout()
10
11     def forward(self, input_ids):
12         seq_len = input_ids.size(1)
13         position_ids = torch.arange(seq_len, dtype=torch.long).unsqueeze(0)
14         token_emb = self.token_embeddings(input_ids)
15         pos_emb = self.position_embeddings(position_ids)
16         embeddings = self.layer_norm(token_emb + pos_emb)
17         return self.dropout(embeddings)

```

Listing 29: Learnable positional embedding module

3.7 Full Transformer Encoder

```

1 class TransformerEncoder(nn.Module):
2     def __init__(self, config):
3         super().__init__()
4         self.embeddings = Embeddings(config)
5         self.layers = nn.ModuleList(
6             [TransformerEncoderLayer(config)
7              for _ in range(config.num_hidden_layers)])
8
9     def forward(self, x):
10         x = self.embeddings(x)
11         for layer in self.layers:

```

```

12         x = layer(x)
13     return x
14 # Output: [batch, seq_len, hidden_dim] e.g. [1, 5, 768]

```

Listing 30: Full transformer encoder (body only)

3.8 Adding a Classification Head

Transformer models separate a **task-independent body** (the encoder stack) from a **task-specific head**.

For sequence classification, use the [CLS] token's hidden state:

```

1 class TransformerForSequenceClassification(nn.Module):
2     def __init__(self, config):
3         super().__init__()
4         self.encoder = TransformerEncoder(config)
5         self.dropout = nn.Dropout(config.hidden_dropout_prob)
6         self.classifier = nn.Linear(config.hidden_size, config.
            num_labels)
7
8     def forward(self, x):
9         x = self.encoder(x)[: , 0, :] # [CLS] hidden state
10        x = self.dropout(x)
11        return self.classifier(x)
12 # Output: [batch, num_labels] e.g. [1, 3] (unnormalised
    logits)

```

Listing 31: Transformer for sequence classification

3.9 The Decoder

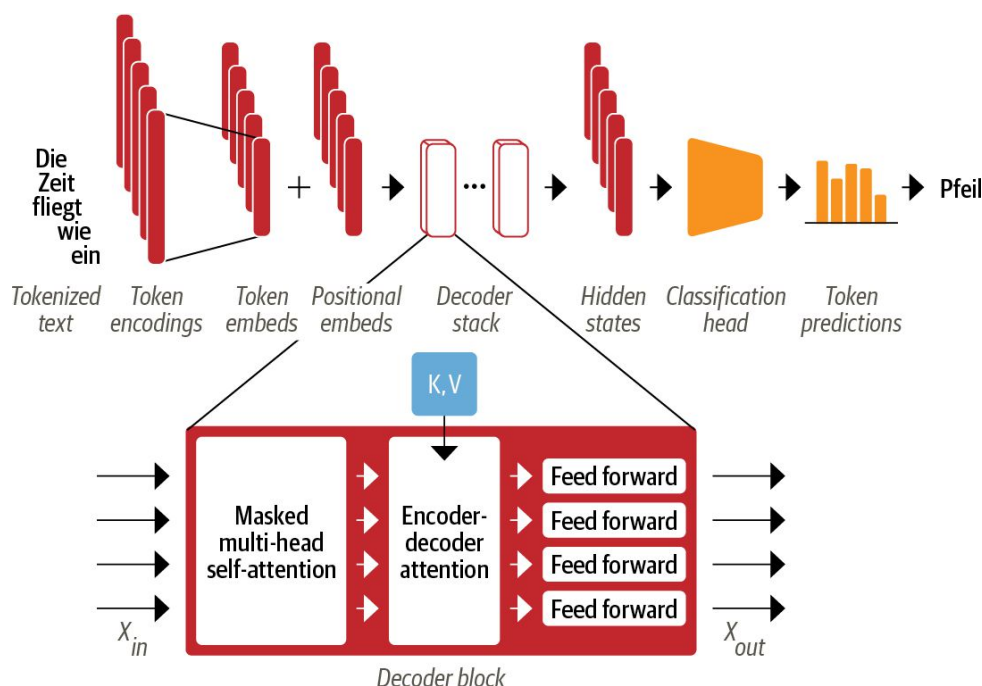


Figure 23: Zooming into the decoder layer. Two extra components vs. the encoder: (1) **masked** multi-head self-attention (prevents attending to future tokens during training); (2) **encoder-decoder attention** (queries from the decoder attend to encoder keys and values).

The decoder has **two attention sublayers** not present in the encoder:

1. **Masked multi-head self-attention** — ensures each generated token only depends on past outputs. Implemented with a lower-triangular mask:

```

1 seq_len = inputs.input_ids.size(-1)
2 mask = torch.tril(torch.ones(seq_len, seq_len)).unsqueeze(0)
3 # Positions above the diagonal get -inf before softmax
4 scores.masked_fill(mask == 0, float("-inf"))

```

Listing 32: Causal mask for the decoder

2. **Encoder-decoder attention** — decoder queries attend to encoder keys and values; allows the decoder to “read” the full source context.

3.10 Interoperability Between Frameworks

```

1 from transformers import TFAutoModel
2 tf_model = TFAutoModel.from_pretrained("distilbert-base-uncased")
3
4 # If only PyTorch weights are available, convert automatically:

```

```
5 tf_xlmr = TFAutoModel.from_pretrained("xlm-roberta-base",
    from_pt=True)
```

Listing 33: Loading a model in TensorFlow instead of PyTorch

The pattern: prefix class names with TF and replace the "pt" format string with "tf" throughout.

PyTorch	TensorFlow (Keras)	Role
nn.Linear	keras.layers.Dense	Dense layer
nn.Module	keras.layers.Layer	Model building block
nn.Dropout	keras.layers.Dropout	Dropout
nn.LayerNorm	keras.layers.LayerNormalization	Layer norm
nn.Embedding	keras.layers.Embedding	Embedding table
nn.GELU	keras.activations.gelu	GELU activation
torch.bmm	tf.matmul	Batched matrix multiply
model.forward	model.call	Forward pass

3.11 The Transformer Family Tree

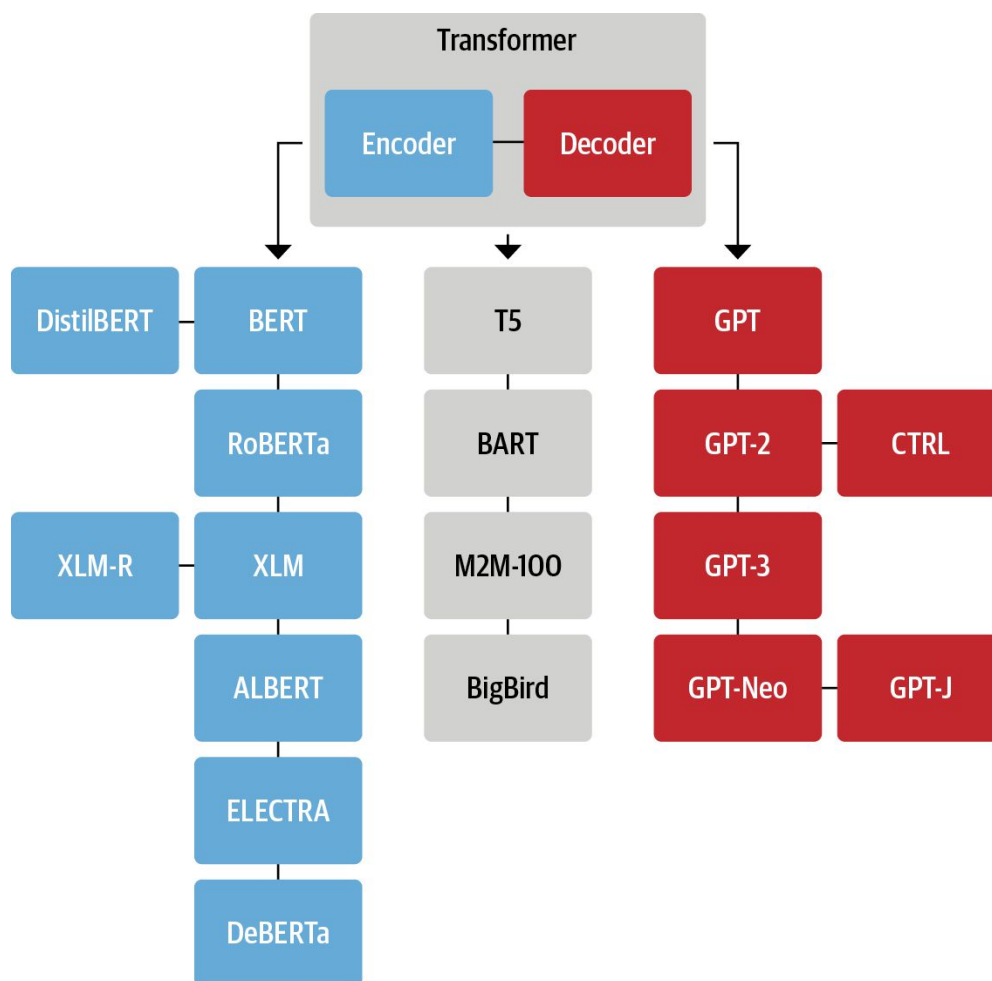


Figure 24: Overview of prominent transformer architectures and their lineage. Blue = encoder-only; Red = decoder-only; Grey = encoder-decoder.

3.11.1 Encoder-Only Branch

Model	Key innovations
BERT	Pretraining with Masked Language Modelling (MLM) + Next Sentence Prediction (NSP) . First model to outperform state-of-the-art on the GLUE benchmark.
DistilBERT	Knowledge distillation of BERT: 97% performance, 40% less memory, 60% faster inference.
RoBERTa	Longer training, larger batches, more data, no NSP objective . Significantly outperforms BERT.

XLM	Multilingual pretraining with MLM + Translation Language Modelling (TLM) . State-of-the-art on multilingual NLU benchmarks.
XLM-R	XLM + RoBERTa ideas; trained on 2.5 TB of multilingual Common Crawl data. Especially strong for low-resource languages.
ALBERT	(1) Factorised embedding parameterisation, (2) cross-layer parameter sharing, (3) Sentence Order Prediction (SOP) instead of NSP. Larger models with fewer parameters.
ELECTRA	Two-model: a <i>generator</i> (MLM) + a <i>discriminator</i> (predicts which tokens were replaced). 30× more efficient pretraining.
DeBERTa	Disentangled content and position representations; absolute position embedding before the output softmax. First model (ensemble) to beat human baseline on SuperGLUE.

3.11.2 Decoder-Only Branch

Model	Key innovations
GPT	Transformer decoder + transfer learning for NLU/NLG. Pretrained on BookCorpus with next-word prediction.
GPT-2	Scaled-up GPT (1.5B parameters); impressive coherent long-form text generation. Released in stages due to misuse concerns.
CTRL	Control tokens prepended to the prompt let the user steer generation style.
GPT-3	175B parameters; follows scaling laws (power-law relations between compute, data, model size, and performance). Strong few-shot learning without gradient updates. Not open-sourced.
GPT-Neo / GPT-J-6B	Open-source GPT-3-scale models (1.3B, 2.7B, 6B) by EleutherAI; competitive with the smaller GPT-3 variants.

3.11.3 Encoder-Decoder Branch

Model	Key innovations
T5	Unifies all NLU & NLG tasks as text-to-text problems. Pretrained with MLM on the C4 corpus; 11B parameter version achieves state-of-the-art on many benchmarks.
BART	Combines BERT-style encoder with GPT-style decoder. Pretraining: reconstruct corrupted text (masking, permutation, deletion, rotation). Excels at both NLU and NLG.
M2M-100	First model to translate directly between any pair of 100 languages without routing through English. Uses language-pair prefix tokens.
BigBird	Uses sparse attention (linear scaling) to extend the context window from 512 to 4,096 tokens; ideal for long-document tasks such as summarisation.

3.12 Chapter 3 Summary — Building Blocks

Component	Role
Token embedding	Maps token IDs to dense vectors.
Positional embedding	Injects order information (learned or fixed).
Scaled dot-product attention	Core similarity computation; $O(n^2)$ complexity.
Multi-head attention	Parallel attention heads capture diverse relationships.
Feed-forward network	Position-wise transformation; main capacity store.
Layer normalisation	Stabilises training; pre-LN placement preferred.
Skip connections	Preserve gradient flow through deep stacks.
Causal mask (decoder)	Prevents attending to future tokens.
Classification head	Task-specific output layer appended to the body.

4 Quick-Reference Cheat Sheet

4.1 Important Hyperparameters for DistilBERT / BERT

Hyperparameter	Value (BERT-base)	Notes
<code>vocab_size</code>	30,522	WordPiece vocabulary
<code>hidden_size</code>	768	Token embedding dimension
<code>num_hidden_layers</code>	12	Encoder layers (6 for DistilBERT)
<code>num_attention_heads</code>	12	
<code>head_dim</code>	64	768/12
<code>intermediate_size</code>	3,072	FFN hidden size (4×768)
<code>max_position_embeddings</code>	512	Maximum sequence length
<code>hidden_dropout_prob</code>	0.1	

4.2 Fine-Tuning Checklist

1. Load dataset with `load_dataset()` from the Hub or local files.
2. Explore class distribution and text lengths.
3. Load matching tokeniser with `AutoTokenizer.from_pretrained()`.
4. Tokenise with `map(tokenize, batched=True)`.
5. Load model with `AutoModelForSequenceClassification.from_pretrained()`.
6. Define `compute_metrics()` (accuracy + F1).
7. Configure `TrainingArguments` (LR, epochs, batch size, eval strategy).
8. Run `trainer.train()`.
9. Analyse errors by sorting validation samples by loss.
10. Push to Hub with `trainer.push_to_hub()`.

4.3 Key Equations

Scaled dot-product attention:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

Multi-head attention:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O, \quad \text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$$

Feed-forward network:

$$\text{FFN}(\mathbf{x}) = \text{GELU}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

Self-attention output (pre-LN encoder layer):

$$\mathbf{x}' = \mathbf{x} + \text{Attention}(\text{LN}(\mathbf{x})), \quad \mathbf{x}'' = \mathbf{x}' + \text{FFN}(\text{LN}(\mathbf{x}'))$$

*Notes compiled from Natural Language Processing with Transformers (Tunstall, von Werra & Wolf, O'Reilly 2022), pages 15–84. Figures reference extracted images from the PDF using the naming convention **Figure X-Y.jpg** / **Figure X-Y-N.jpg**. Place all image files in the same directory as this `.tex` file before compiling.*